

Software Development Project Report

On

Personalized Chatbot to Entertain the Users

Prepared By:

Sr. No.	Name	ID
1	Moradiya Tilak Rajeshbhai	23CEUOS168
2	Kachariya Smit Sanjaybhai	23CEUBS063

B. Tech. CE Semester – VI
Subject: System Design Practise



Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University



**Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University**

Certificate

This is to certify that the project entitled "Personalized Chatbot to Entertain the Users" has been successfully completed by the following students of B.Tech Semester VI, Computer Engineering, Dharmsinh Desai University, Nadiad.

Sr. No.	Name	ID
1	Moradiya Tilak Rajeshbhai	23CEUOS168
2	Kachariya Smit Sanjaybhai	23CEUBS063

The project has been carried out to our satisfaction and is approved for the partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Engineering during the academic year 2025-26.

Date: _____

Dr. Brijesh S. Bhatt

Professor,
Dept. of Computer Engineering,
Faculty of Technology,
Dharmsinh Desai University, Nadiad

Dr. C. K. Bhensdadia

Head of the Department,
Dept. of Computer Engineering,
Faculty of Technology,
Dharmsinh Desai University, Nadiad

Table of Contents

1	Introduction	7
1.1	Brief Introduction	7
1.2	Tools, Technology and Platform Used	7
2	Software Requirement Specifications	9
2.1	Product Scope	9
2.2	Types of Users	9
2.3	System Functional Requirements	9
2.4	Other Non-functional Requirements	10
3	Design	12
3.1	Use Case Diagram	12
3.2	Class Diagrams	13
3.3	State Diagram	16
3.4	Activity Diagrams	17
3.5	Sequence Diagrams	20
4	Implementation Details	21
4.1	Front-End Design Implementation Details	21
4.2	Back-End Implementation Details	22
4.3	Intelligent Response System	24
4.4	Game Features Implementation	25
4.5	Spell Correction Implementation	25
5	Testing	26
5.1	Testing Method Used	26
5.2	Test Cases	26
6	Screenshots	28
7	Conclusion	39
8	Limitation and Future Extension	40
9	Bibliography	

List of Figures

Figure No.	Figure
3.1	Use Case Diagram
3.2.1	User & Chat Class Diagram
3.2.2	Intelligent Response System Class Diagram
3.2.3	Game Handler Class Diagram
3.3	State Diagram
3.4.1	Chat Processing Activity Diagram
3.4.2	Game Session Activity Diagram
3.4.3	Streak Tracking Activity Diagram
3.5.1	Chat Message Sequence Diagram
3.5.2	Authentication Sequence Diagram
4.1	React App's Folder Structure
4.2	Flask Backend Architecture
4.3	5-Tier Fallback Response System
6.1	Login Screen
6.2	Signup Screen
6.3	Dashboard - Welcome Screen
6.4	Chat Conversation
6.5	Game Session (Quiz)
6.6	Profile Page with Streak Stats
6.7	Activity Calendar Heatmap
6.8	Notification Panel
6.9	Milestone Celebration
6.10	Theme Selection
6.11	Voice Settings

List of Tables

Table No.	Table
4.1	Database Model Summary
4.2	API Endpoints Summary
4.3	Intent Classification Categories
4.4	Entity Extraction Categories
5.1	Test Cases for Chat Response System
5.2	Test Cases for Game Features
5.3	Test Cases for Spell Correction
5.4	Test Cases for Authentication
5.5	Test Cases for Streak Tracking
5.6	Test Cases for Voice Features
9.1	References for NLP and AI
9.2	References for Frontend Development
9.3	References for Backend Development

Abstract

"The best way to predict the future is to create it." -- Abraham Lincoln

The Personalized Chatbot to Entertain the Users is a web-based conversational application that provides emotionally intelligent and personalized responses to users. Unlike traditional chatbots that rely on external Large Language Models (LLMs), this chatbot uses a fully custom-built Natural Language Processing (NLP) pipeline that combines intent classification, entity extraction, dynamic response generation, and semantic search using FAISS vector indexing -- all running locally without any API calls to third-party AI services.

The system is designed to understand user emotions, provide empathetic responses, offer advice on topics such as school, work, relationships, health, and finances, and engage users through interactive games including quizzes, riddles, word scrambles, and trivia. It also features a comprehensive engagement tracking system with daily streak monitoring, an activity calendar heatmap, milestone celebrations, and a smart notification system.

The application follows a three-tier architecture with a React 18 frontend, a Flask REST API backend, and a PostgreSQL database. Additional features include voice input/output using the Web Speech API, automatic spell correction, six customizable color themes, persistent chat history, and user profile management with birthday tracking.

1. Introduction

1.1 Brief Introduction

The Personalized Chatbot to Entertain the Users is a full-stack web application designed to serve as a personalized, emotionally aware conversational partner. The project's primary aim is to provide users with an entertaining space to express their feelings, seek advice, and engage in recreational activities -- all through a highly interactive chat interface.

In this system, users can interact with the chatbot in multiple ways:

1. Emotional Conversations -- Share feelings and receive empathetic, contextually appropriate responses
2. Advice Seeking -- Get topic-specific advice about school, work, relationships, health, money, and future planning
3. Interactive Games -- Play quizzes (general, science, programming), riddles, word scrambles, and trivia
4. Voice Interaction -- Use speech-to-text for input and text-to-speech for bot responses
5. Engagement Tracking -- Track daily usage streaks, view activity on a GitHub-style calendar heatmap, and celebrate milestones

The chatbot's intelligence is powered by a multi-layered NLP pipeline that classifies user intent, extracts emotions and topics, generates dynamic template-based responses, and falls back to a semantic search-powered knowledge base indexed using FAISS (Facebook AI Similarity Search) with sentence-transformer embeddings.

1.2 Tools, Technology and Platform Used

Backend:

1. Programming Language: Python 3.14
2. Web Framework: Flask 3.0.0
3. ORM: SQLAlchemy >= 2.0.36 with Flask-SQLAlchemy 3.1.1
4. Database: PostgreSQL (via psycopg >= 3.2.0)
5. Authentication: Flask-JWT-Extended 4.6.0, Flask-Bcrypt 1.0.1
6. NLP/ML: sentence-transformers (all-MiniLM-L6-v2), FAISS (faiss-cpu), NumPy
7. Spell Checking: pypellchecker 0.8.4
8. Environment: python-dotenv 1.0.0
9. CORS: Flask-CORS 4.0.0

Frontend:

1. Programming Language: JavaScript (ES6+)
2. UI Framework: React 18.2.0 with React DOM
3. Routing: React Router DOM 6.20.0
4. HTTP Client: Axios 1.6.2
5. Build Tool: Create React App (react-scripts 5.0.1)
6. Voice: Web Speech API (SpeechRecognition & SpeechSynthesis) -- browser-native

7. Notifications: Web Notifications API -- browser-native

Development Tools:

1. IDE: Visual Studio Code
2. Version Control: Git & GitHub
3. Package Managers: pip (Python), npm (Node.js)

2. Software Requirement Specifications

2.1 Product Scope

The Personalized Chatbot to Entertain the Users is a web-based application designed to provide personalized, emotionally intelligent conversational experiences. The system integrates natural language processing, semantic search, game logic, voice I/O, and engagement tracking into a cohesive user-facing product. The scope of the system is accessible to any user with a modern web browser and internet connectivity.

The application provides the following core functionalities:

- Emotionally aware chat responses with intent classification and entity extraction
- Persistent multi-session chat history with sidebar navigation
- Interactive games (quiz, riddle, word scramble, trivia, jokes)
- Voice input (speech-to-text) and voice output (text-to-speech)
- Daily streak tracking with milestone celebrations
- Activity calendar heatmap visualization
- Smart notification system (in-app, toast, desktop)
- User profile management with birthday tracking
- Six customizable color themes
- Automatic spell correction with slang awareness

2.2 Types of Users

The system has two types of end-users:

- 1. Unregistered User** -- Can view the login/signup page only.
- 2. Registered User** -- Can access all features including chat, games, voice, profile, streak tracking, notifications, and theme customization after authentication.

2.3 System Functional Requirements

R1. User Authentication

Description: Users must be able to register with email, username, and password. Existing users can log in with email and password. Passwords are securely hashed using bcrypt. Sessions are managed using JWT tokens with 24-hour expiry.

R2. Chat Conversation

Description: Users can create multiple chat sessions, send text messages, and receive emotionally intelligent responses. The system maintains conversation history and supports contextual follow-ups.

R3. Intelligent Response System

Description: The bot uses a multi-stage NLP pipeline to classify intent (greeting, farewell, gratitude, seeking advice, sharing feelings, asking questions), extract entities (topics, emotions, people, behaviors), and generate dynamic responses with template formatting and knowledge base fallback.

R4. Spell Correction

Description: Every user message is automatically spell-checked using pyspellchecker. Common slang and abbreviations (50+ words) are preserved. Corrections are displayed in the chat UI.

R5. Interactive Games

Description: Users can play four types of games within the chat: Quiz (3 categories, MCQ format), Riddle (with hints), Word Scramble (with hints), and Trivia (random facts). Additionally, users can request jokes. Game state is persisted per chat session.

R6. Voice Input and Output

Description: Users can input messages via speech recognition (Web Speech API) and have bot responses read aloud via speech synthesis. Configurable voice selection, speech rate, and pitch.

R7. Streak and Activity Tracking

Description: The system tracks daily user engagement. Consecutive active days form a streak with milestones at 3, 7, 14, 30, 60, and 100 days. An activity calendar shows 6 months of engagement data with 5 intensity levels.

R8. Notification System

Description: The system generates real-time notifications including birthday wishes, streak milestone alerts, daily streak reminders, and time-based greetings. Notifications are displayed as in-app toasts, a panel dropdown, and desktop notifications.

R9. User Profile Management

Description: Users can view and edit their profile including username and birthday. The profile page displays streak statistics, achievements, and the activity calendar.

R10. Theme Customization

Description: Users can select from 6 gradient color themes. The selected theme is persisted in localStorage and applied globally using CSS custom properties.

2.4 Other Non-functional Requirements

1) Performance

- The application should provide chatbot responses within 2 seconds.
- Semantic search using FAISS should return results efficiently even with a large knowledge base.
- The activity calendar and streak data should load without noticeable delay.

2) Reliability

- The system must handle database transactions with proper rollback on errors.
- JWT tokens must expire after 24 hours for security.
- Game state must be preserved across page refreshes within the same chat session.

3) Usability

- The UI must be responsive and user-friendly with smooth animations.
- Voice input/output must work seamlessly with browser-native APIs.
- Notifications must not be intrusive -- seen notifications should not reappear.

4) Security

- Passwords must be hashed using bcrypt before storage.
- All protected API endpoints must require valid JWT authentication.
- Cross-Origin Resource Sharing (CORS) must be configured properly.

3. Design

3.1 Use Case Diagram

The use case diagram illustrates the interactions between user types and the system's functionalities.

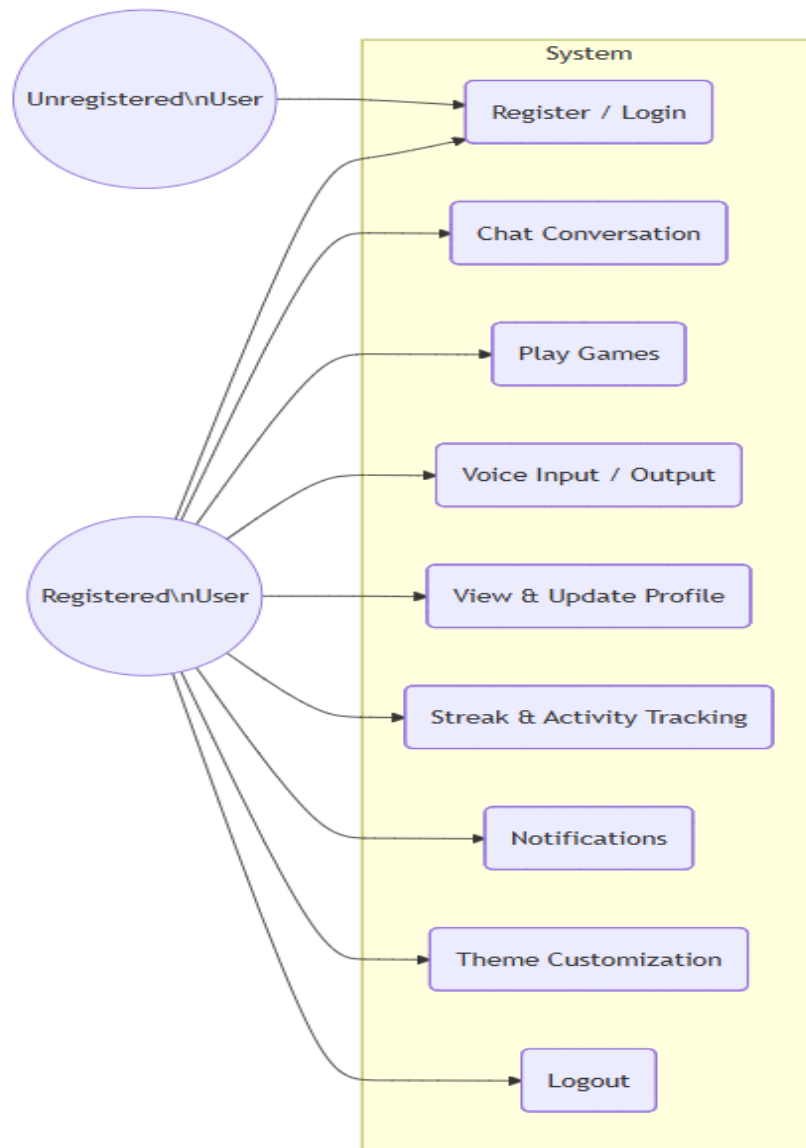


Fig. 3.1: Use Case Diagram

3.2 Class Diagrams

3.2.1 User & Chat Class Diagram

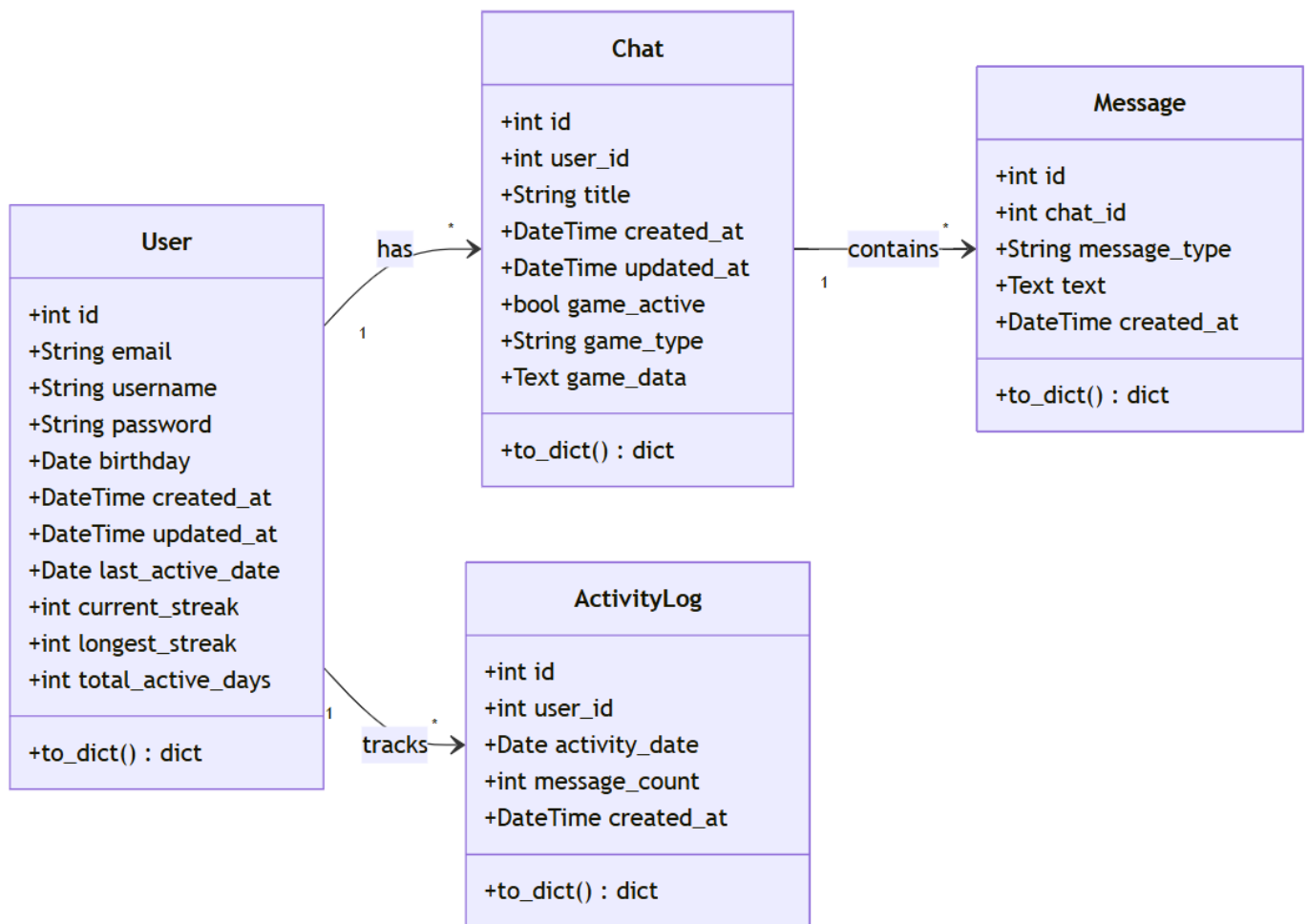


Fig. 3.2.1: User & Chat Class Diagram

3.2.2 Intelligent Response System Class Diagram

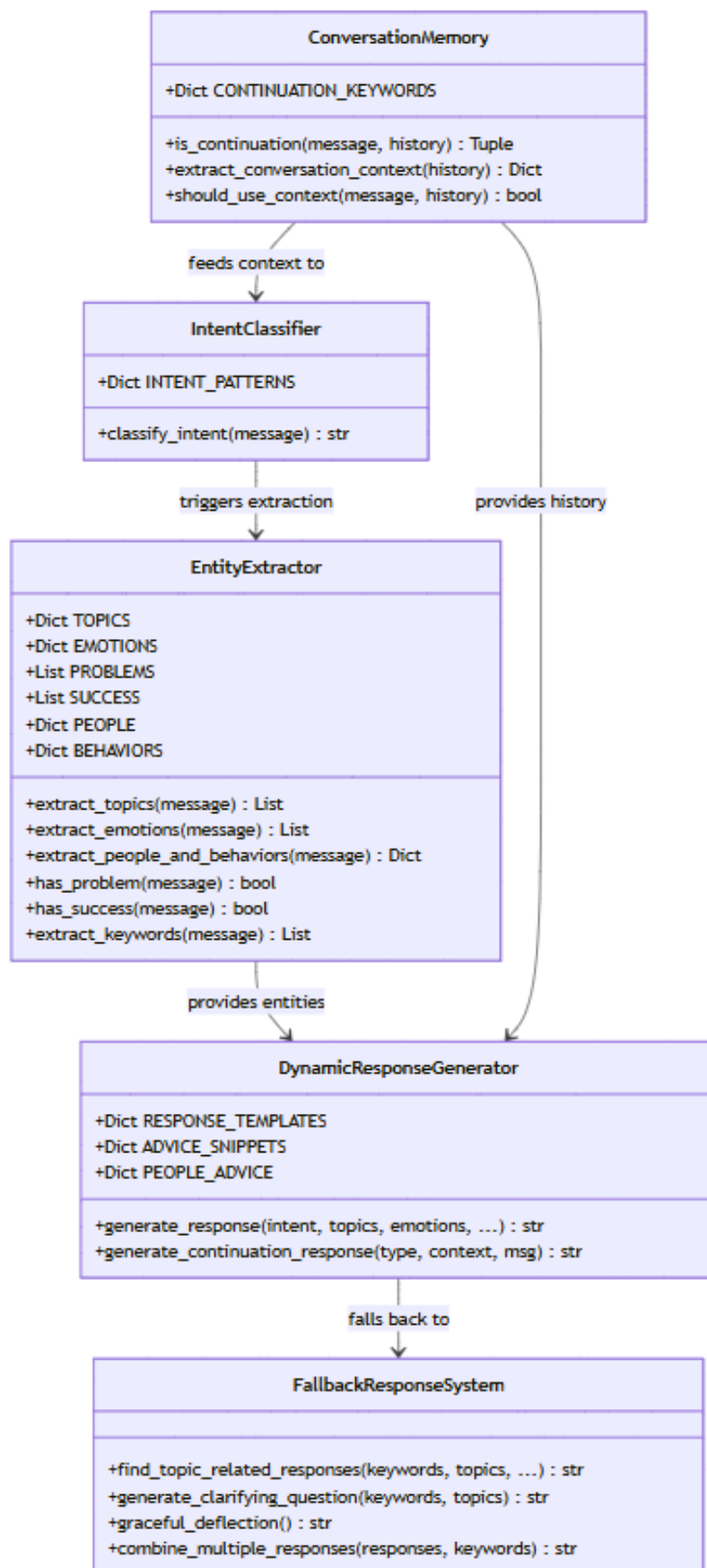


Fig. 3.2.2: Intelligent Response System Class Diagram

3.2.3 Game Handler Class Diagram

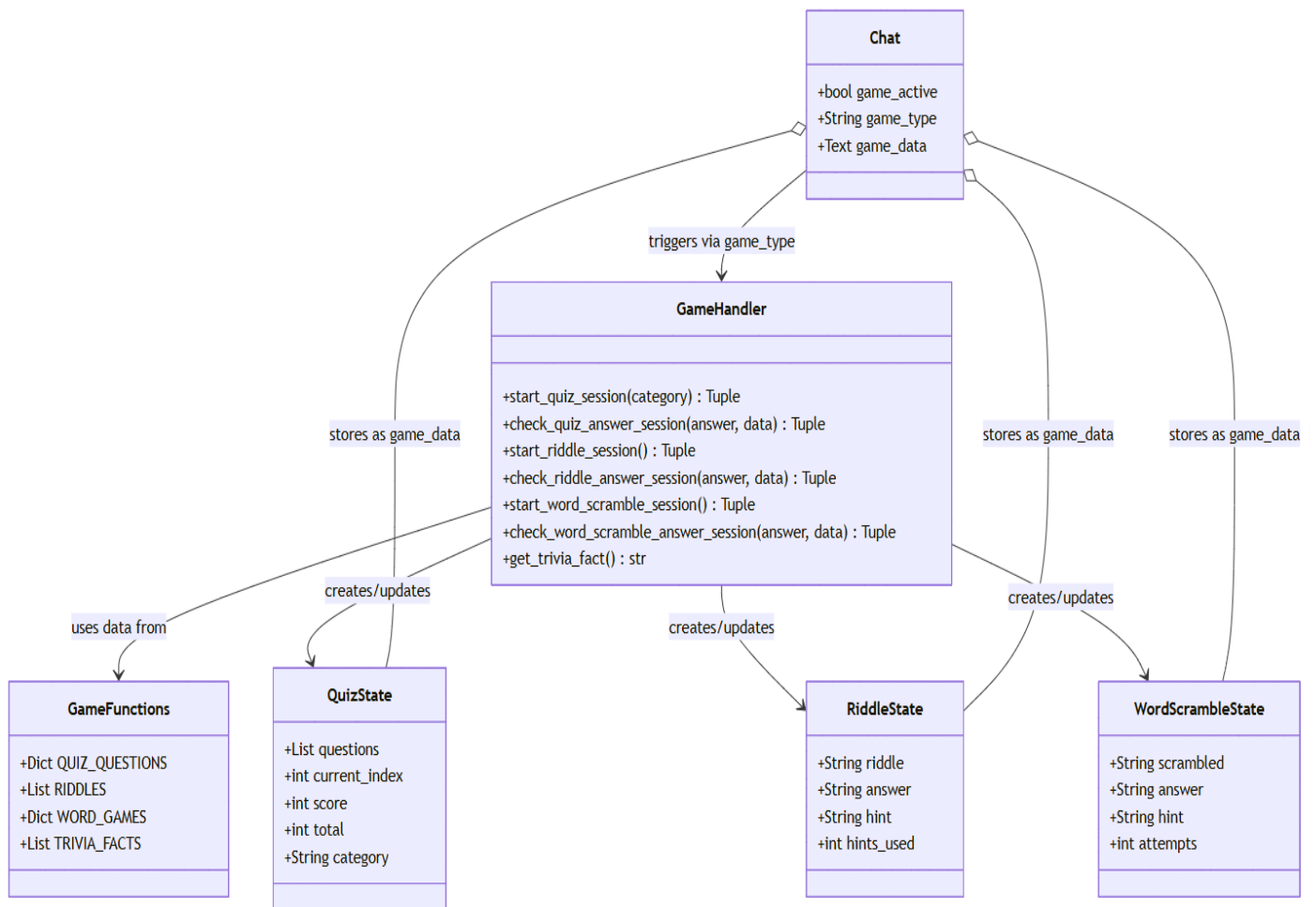


Fig. 3.2.3: Game Handler Class Diagram

3.3 State Diagram

The state diagram shows the various states the application transitions through during user interaction.

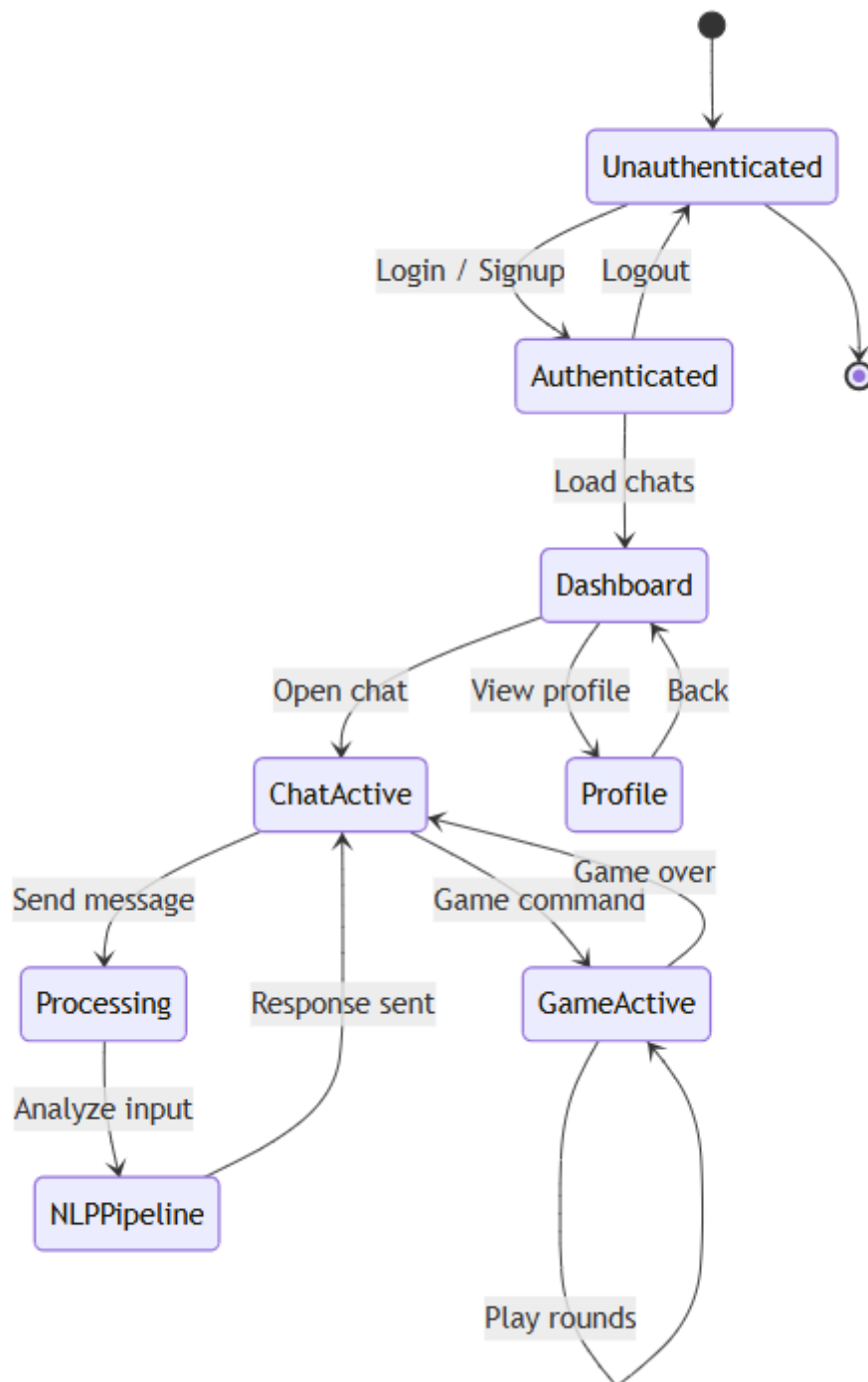


Fig. 3.3: State Diagram

3.4 Activity Diagrams

3.4.1 Chat Processing Activity Diagram

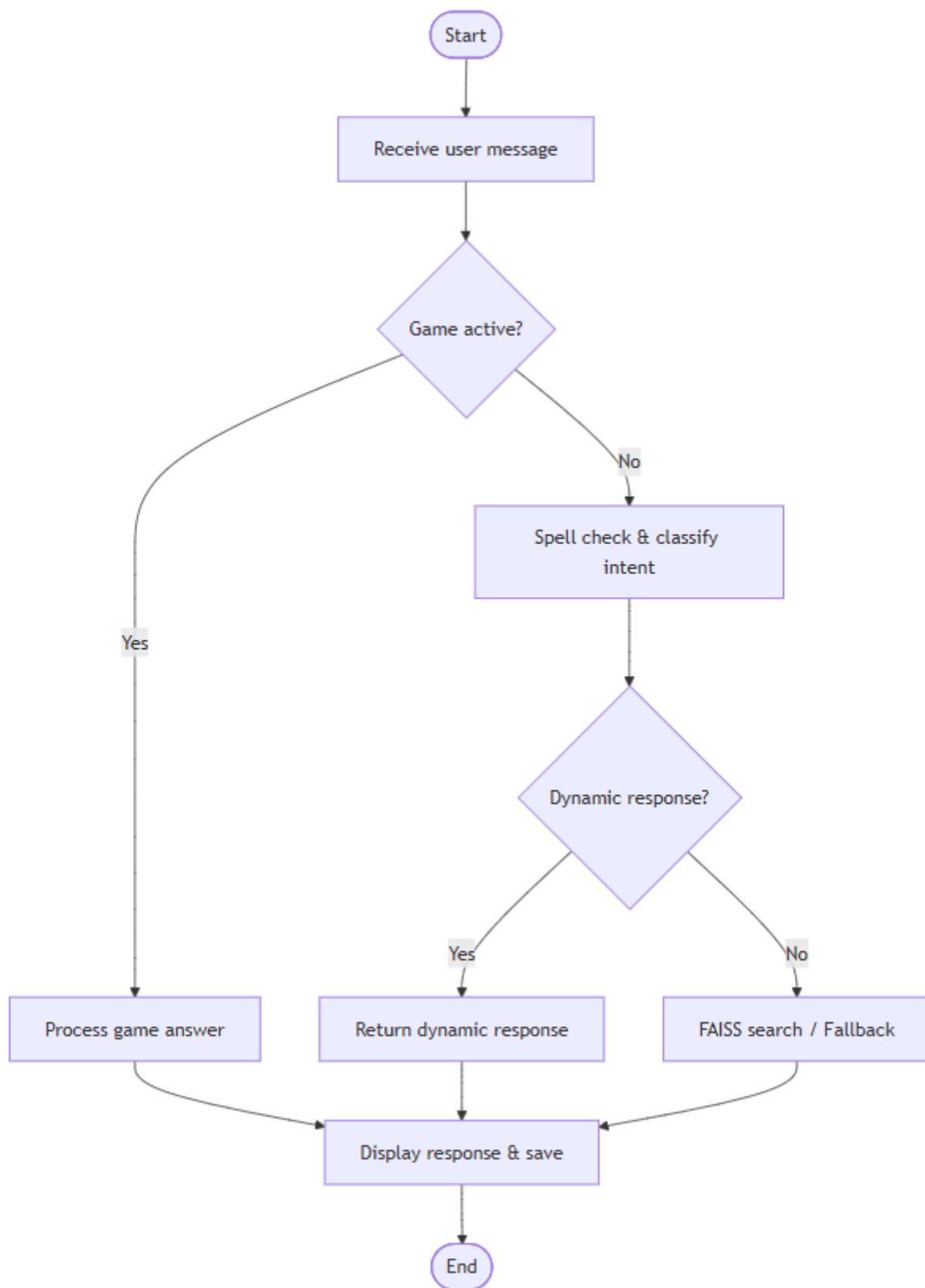


Fig. 3.4.1: Chat Processing Activity Diagram

3.4.2 Game Session Activity Diagram

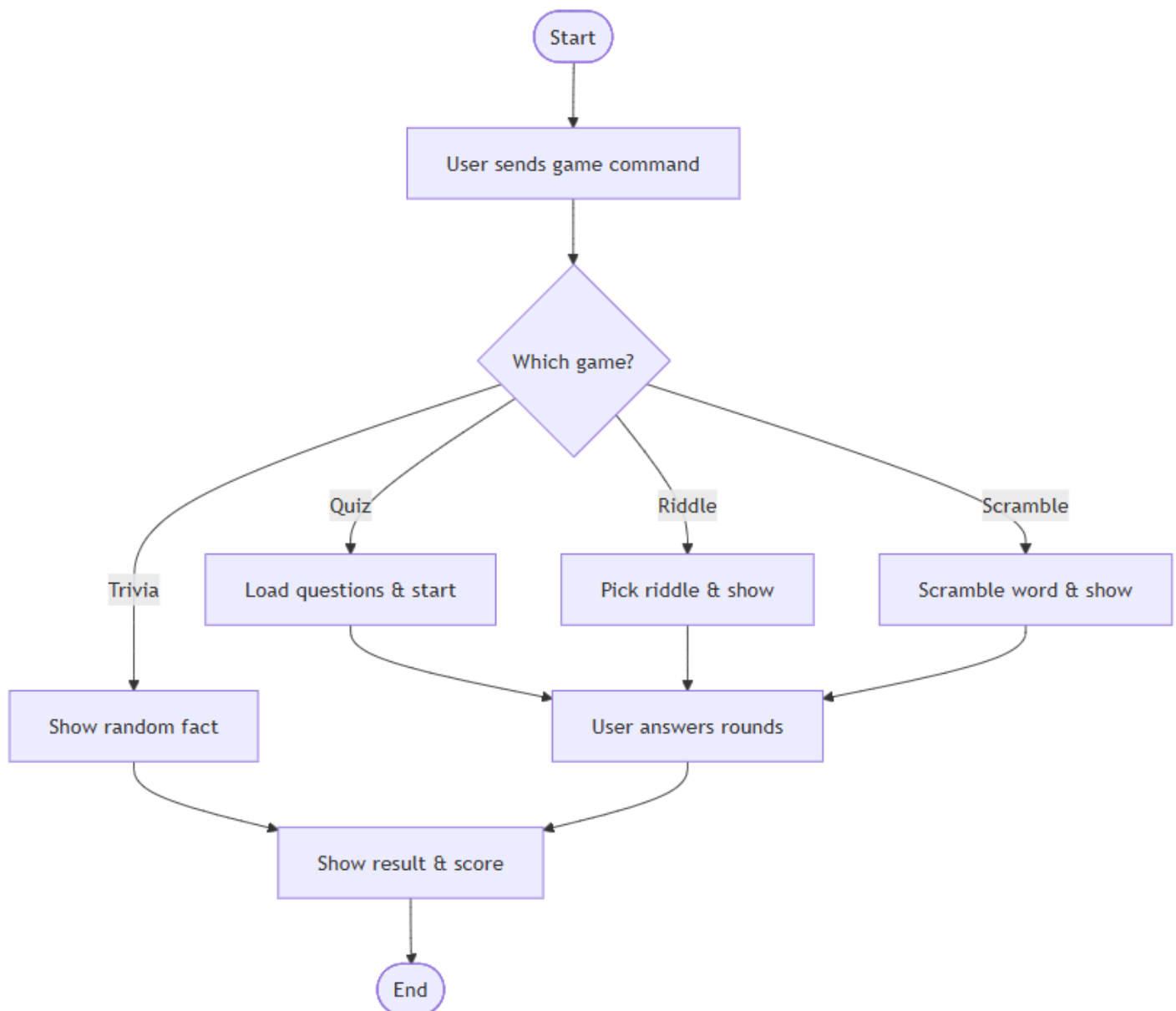


Fig. 3.4.3: Game Session Activity Diagram

3.4.5 Streak Tracking Activity Diagram

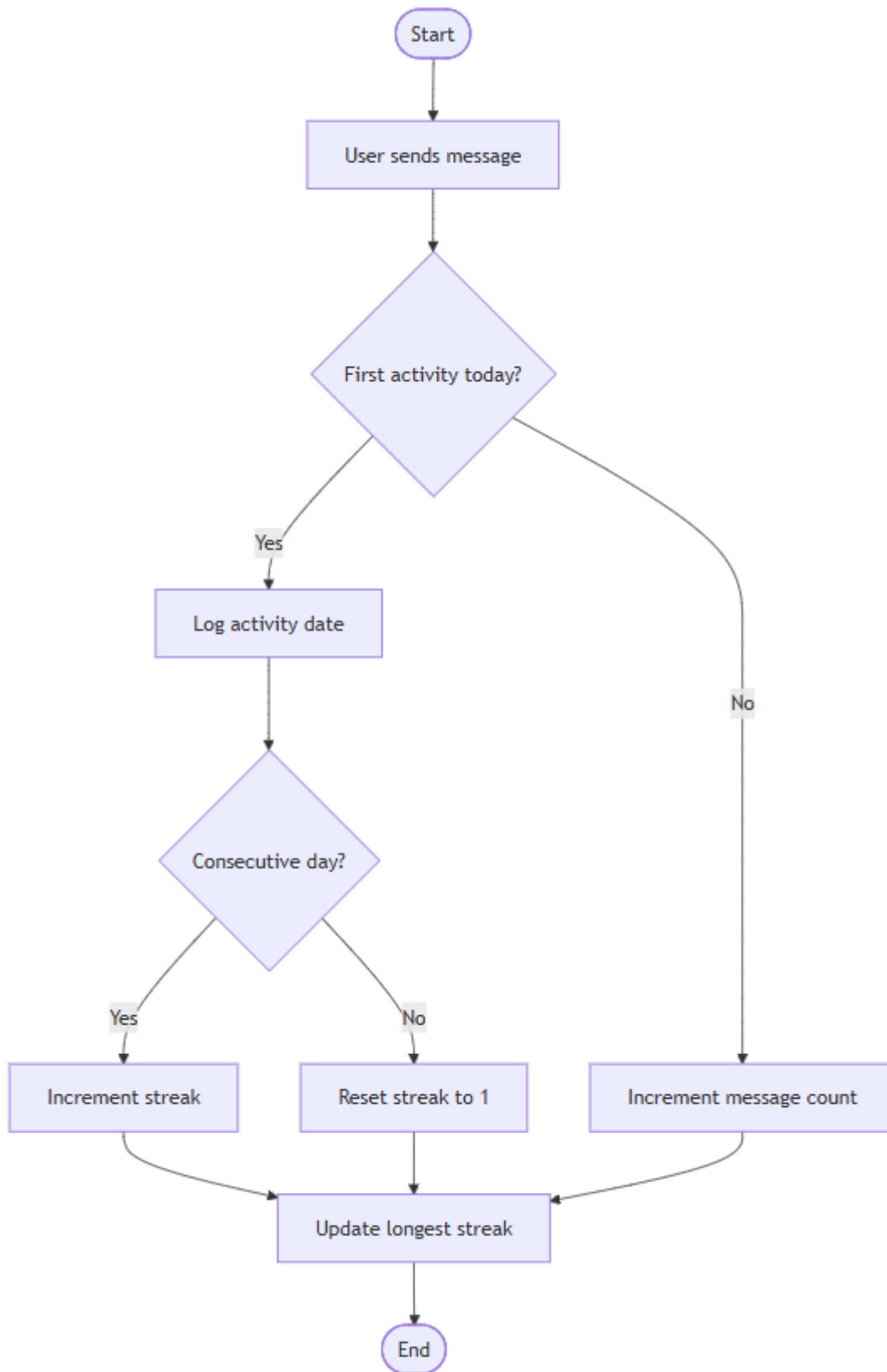


Fig. 3.4.5: Streak Tracking Activity Diagram

3.5 Sequence Diagrams

3.5.1 Chat Message Sequence Diagram

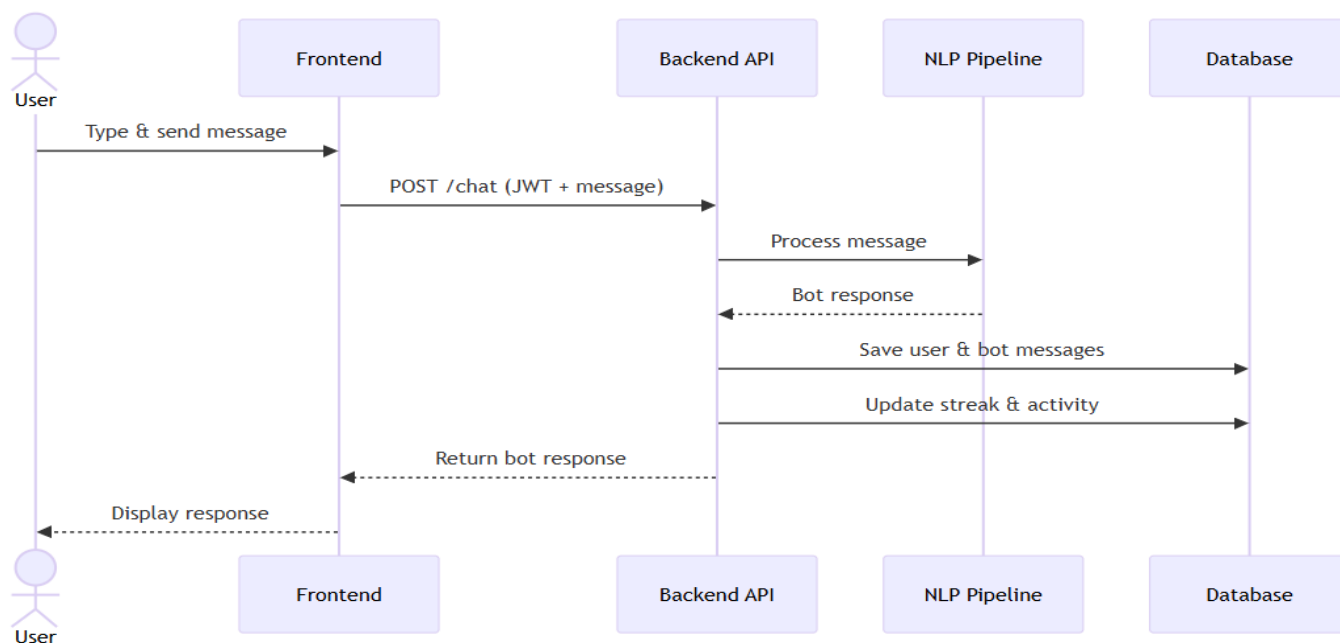


Fig. 3.5.1: Chat Message Sequence Diagram

3.5.2 Authentication Sequence Diagram

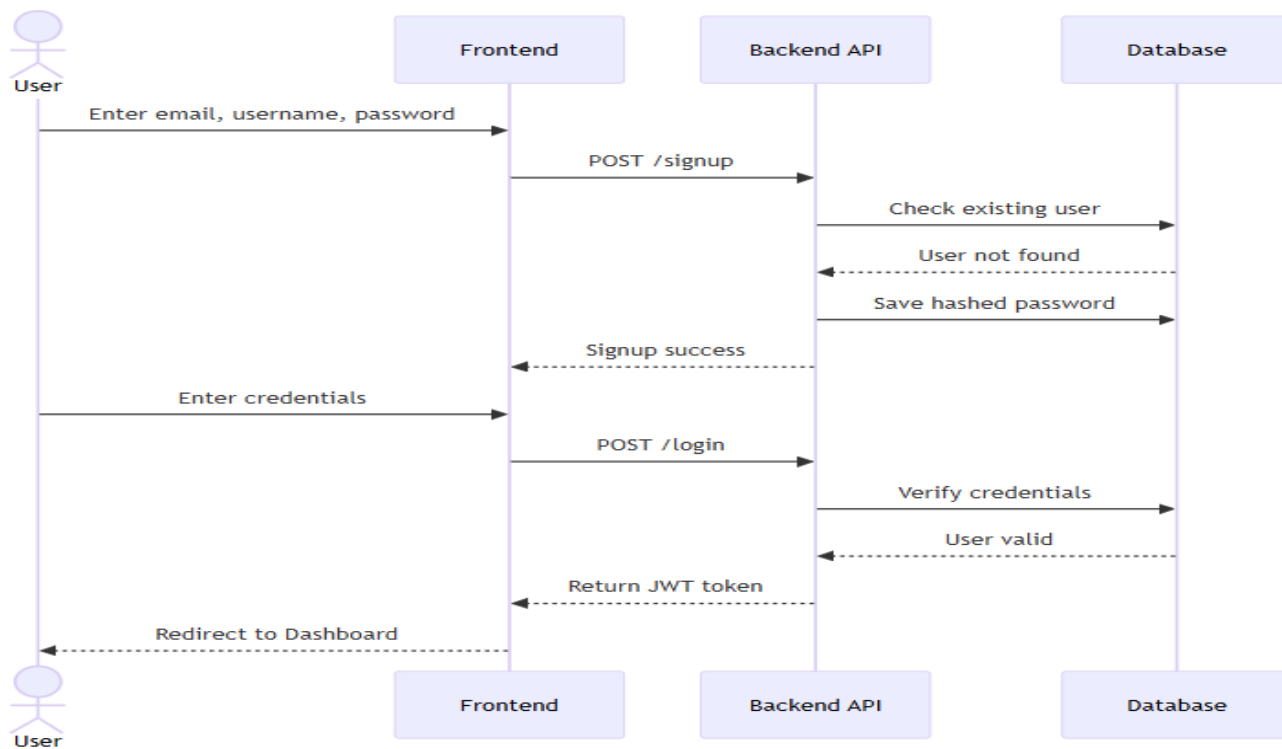


Fig. 3.5.2: Authentication Sequence Diagram

4. Implementation Details

4.1 Front-End Design Implementation Details

The frontend is built using React 18 with functional components and hooks. The application follows a component-based architecture with Context API for global state management.

React App's Folder Structure:

```
frontend/ +-- public/ | +--
index.html
+-- src/
| +-- api/ | | +-- auth.js      <- Axios instance with JWT interceptor
| +-- components/
| | +-- ProtectedRoute.js <- Auth guard component
| | +-- StreakBadge.js    <- Header streak display
| | +-- MilestoneCelebration.js <- Confetti celebration modal
| | +-- ActivityCalendar.js <- GitHub-style heatmap | | +-- NotificationPanel.js <-
Notification dropdown | | +-- NotificationToast.js <- Toast popup
| +-- context/
| | +-- AuthContext.js    <- Authentication state
| | +-- ThemeContext.js   <- Theme management | | +-- VoiceContext.js    <- Speech
recognition & synthesis
| +-- pages/
| | +-- Login.js, Signup.js <- Auth pages
| | +-- Dashboard.js      <- Main chat interface | | +-- Profile.js      <- User
profile & settings
| +-- services/
| | +-- notificationService.js <- Desktop notifications
| +-- App.js              <- Root with routing | +-- index.js          <- Entry
point
+-- package.json
```

Fig. 4.1: React App's Folder Structure

The frontend has the following key views:

- Login Page:** A clean form with email and password fields, gradient-themed background, and links to the signup page. Uses AuthContext for login state management.
- Signup Page:** Registration form with email, username, password, and confirm password fields. Includes client-side validation (min 6 chars, password match) before API submission.
- Dashboard Page:** The main interface containing a header with streak badge and notification bell, sidebar with chat history and new-chat button, chat area with message bubbles and typing indicators, input area with voice button and textarea, and overlay panels for notifications and milestone celebrations.
- Profile Page:** Contains user information, streak statistics in a 2x2 grid, achievements section, activity calendar heatmap, and settings modals for theme picker, voice settings, and birthday picker.

4.2 Back-End Implementation Details

The backend is built using Flask 3.0 as a REST API server with PostgreSQL as the database, accessed through SQLAlchemy ORM.

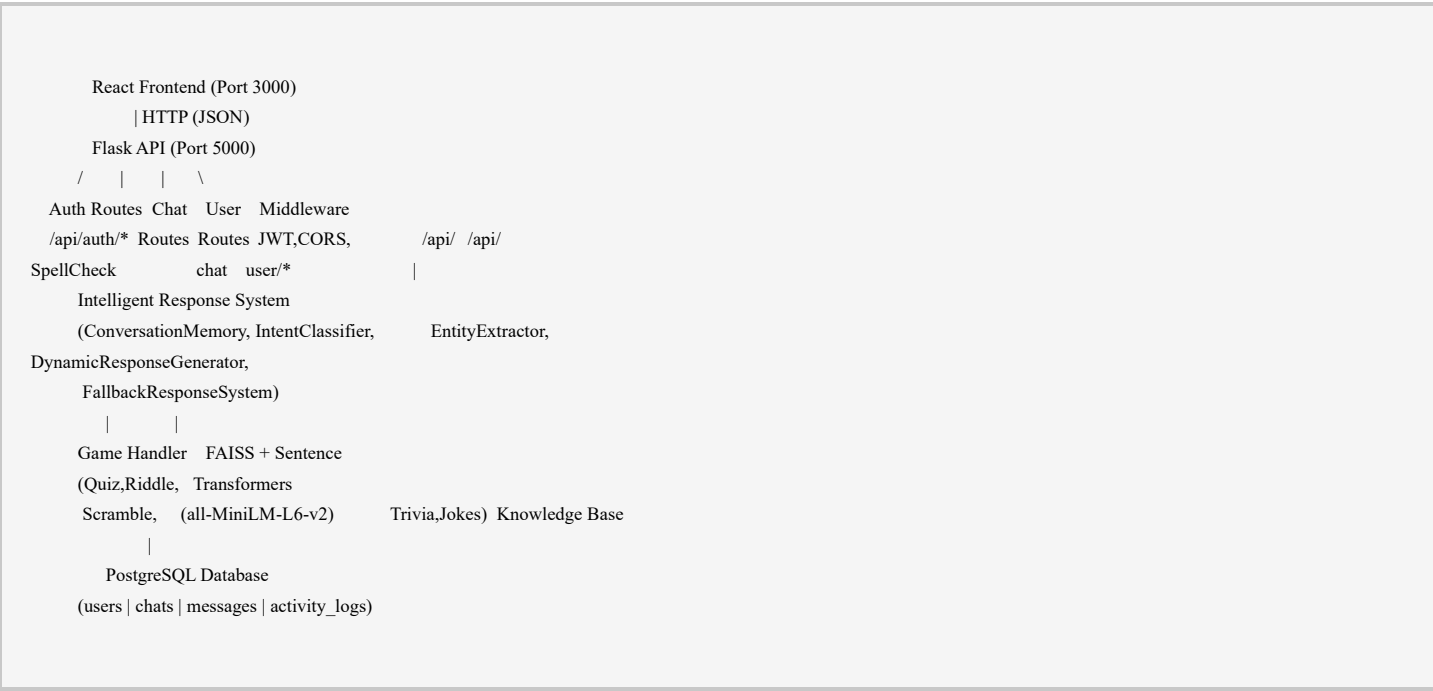


Fig. 4.2: Flask Backend Architecture

Database Models Summary:

Model	Table	Key Fields	Purpose
User	users	id, email, username, password (bcrypt), birthday, streak fields	Auth & streak tracking
Chat	chats	id, user_id, title, game_active, game_type, game_data	Chat sessions with game state
Message	messages	id, chat_id, message_type, text	Individual chat messages
ActivityLog	activity_logs	id, user_id, activity_date, message_count	Daily engagement tracking

Table 4.1: Database Models Summary

API Endpoints Summary:

Method	Route	Auth	Purpose
GET	/api/health	No	Health check
POST	/api/auth/signup	No	Register new user
POST	/api/auth/login	No	Login with credentials
GET	/api/auth/me	JWT	Get current user profile
POST	/api/auth/logout	JWT	Logout user
GET	/api/user/streak	JWT	Get streak statistics
GET	/api/user/activity	JWT	Get 6-month activity data
GET	/api/user/notifications	JWT	Get notification list
PUT	/api/user/profile	JWT	Update profile (birthday, username)
GET	/api/chats	JWT	List all chat sessions
POST	/api/chats	JWT	Create new chat session

DELETE	/api/chats/:id	JWT	Delete a chat session
POST	/api/chat	JWT	Main chat endpoint

Table 4.2: API Endpoints Summary

4.3 Intelligent Response System

The chatbot's intelligence is powered by a 100% local NLP pipeline with no external LLM API calls. The system is implemented in `intelligent_response.py` with four core classes.

Intent Classification:

The `IntentClassifier` uses regex patterns and keyword matching to classify messages into one of 7 intent types:

Intent	Example Messages
greeting	"hello", "hi", "good morning"
farewell	"bye", "goodbye", "see you"
gratitude	"thanks", "thank you", "appreciate it"
seeking_advice	"what should I do", "help me", "need advice"
sharing_feelings	"I feel sad", "I'm stressed", "feeling happy"
asking_question	"what is", "tell me about", "why"
making_statement	(default -- anything else)

Table 4.3: Intent Classification Categories

Entity Extraction:

Entity Type	Categories
Topics	school, work, relationship, health, money, future
Emotions	sadness, stress, anger, happiness, fatigue, boredom
People	teacher, boss, parent, friend, partner, colleague
Behaviors	rude, angry, unfair, ignoring
Indicators	problem (15 keywords), success (14 keywords)

Table 4.4: Entity Extraction Categories

5-Tier Fallback Response System:

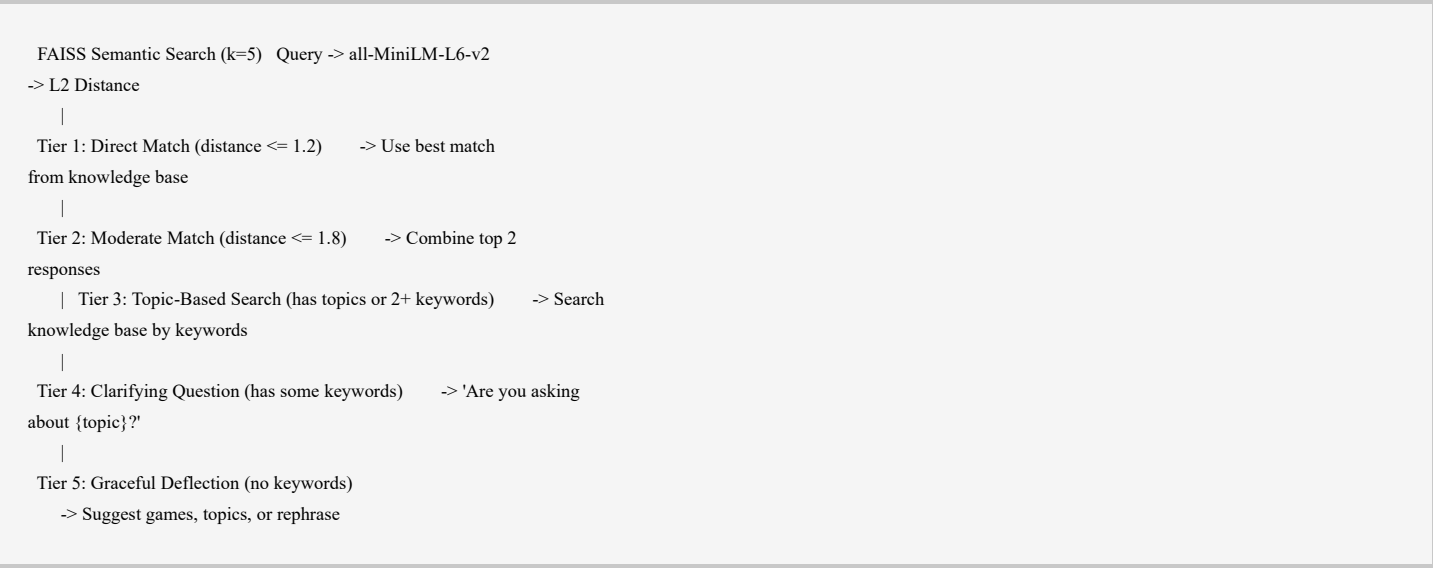


Fig. 4.3: 5-Tier Fallback Response System

4.4 Game Features Implementation

Game state is stored as JSON in the Chat model, allowing per-chat independent game sessions.

Game	Trigger	Mechanics
Quiz	"quiz", "science quiz"	5 MCQs (A/B/C/D), 3 categories, explanations, % scoring
Riddle	"riddle", "brain teaser"	5 riddles, hint system, give-up detection, no-hint bonus
Word Scramble	"word scramble"	5 scrambled words, hints, attempt tracking, first-try bonus
Trivia	"trivia", "fun fact"	8 random facts, no persistent state
Jokes	"joke", "tell me a joke"	20 built-in jokes, random selection

Mood-Escape Feature: If a user expresses emotions or changes intent during an active game (e.g., says "I feel sad" while playing quiz), the system automatically exits the game and routes to the emotional response system.

4.5 Spell Correction Implementation

The spell correction system uses pypellchecker 0.8.4 and processes every incoming user message:

1. Split message into words
2. Skip common abbreviations and slang (50+ entries: "lol", "idk", "omg", "hii", "pls", etc.)
3. Skip words with ≤ 2 characters
4. Preserve leading/trailing punctuation
5. Check each word against the dictionary
6. Apply correction if confident match found
7. Return original + corrected message + list of changes
8. Frontend displays: "Interpreted as: corrected message" above bot response

5. Testing

5.1 Testing Method Used

We have performed unit testing during development and black box testing for validation. Each feature module has been tested with specific test cases covering normal, boundary, and error scenarios.

5.2 Test Cases

5.2.1 Chat Response System

ID	Input	Expected	Actual	Result
T1	"hello"	Greeting response	Hello! How are you today?	Pass
T2	"I feel sad"	Empathetic response	I'm sorry you're feeling this way...	Pass
T3	"my teacher is rude"	Advice about rude teacher	Dealing with a rude teacher is tough...	Pass
T4	"work stress"	Work + stress advice	Work stress is real. Set boundaries...	Pass
T5	"thanks"	Gratitude response	Knowledge base gratitude message	Pass
T6	"bye"	Farewell response	Knowledge base farewell message	Pass
T7	"yes" (continuation)	Context follow-up	Glad you want to talk about it...	Pass
T8	"asdfghjkl"	Graceful deflection	I'm not sure I understood that...	Pass

Table 5.1: Test Cases for Chat Response System

5.2.2 Game Features

ID	Input	Expected	Actual	Result
T1	"quiz"	Start quiz MCQ	Quiz Time! Question 1/5:...	Pass
T2	"A" (quiz)	Check answer	Correct! +10 points...	Pass
T3	"riddle"	Start riddle	Riddle Time! Here's your riddle...	Pass
T4	"hint" (riddle)	Show hint	Hint: [relevant hint]	Pass
T5	"word scramble"	Start scramble	Word Scramble! Unscramble: ELPPA	Pass
T6	"apple" (answer)	Correct result	Correct! The word was APPLE!	Pass
T7	"trivia"	Random fact	Did you know? [random fact]	Pass
T8	"joke"	Random joke	Why don't scientists trust atoms?...	Pass
T9	"I feel sad" (game)	Exit game	Game exits + emotional support	Pass
T10	"quit" (game)	End game	Game ended. New game?	Pass

Table 5.2: Test Cases for Game Features

5.2.3 Spell Correction

ID	Input	Expected	Actual	Result
T1	"happy"	"happy"	"happy" (corrected)	Pass
T2	"hii"	No correction (slang)	"hii" (no correction)	Pass
T3	"lol"	No correction (abbr)	"lol" (no correction)	Pass
T4	"teh"	"the"	"the" (corrected)	Pass
T5	"feelng sad"	"feeling sad"	"feeling sad"	Pass
T6	"amazng"	"amazing"	"amazing"	Pass

Table 5.3: Test Cases for Spell Correction

5.2.4 Authentication

ID	Scenario	Expected	Actual	Result
T1	Valid signup	Account created, JWT	Success, dashboard	Pass
T2	Duplicate email	Error: exists	Email already registered	Pass
T3	Valid login	JWT returned	Success	Pass
T4	Wrong password	Error: invalid	Invalid email or password	Pass
T5	No token access	Redirect login	Redirected to /login	Pass
T6	Short password	Validation error	Min 6 characters error	Pass

Table 5.4: Test Cases for Authentication

5.2.5 Streak Tracking

ID	Scenario	Expected	Actual	Result
T1	First message	Streak = 1	Streak updated	Pass
T2	Consecutive day	Streak +1	current_streak increased	Pass
T3	Skip one day	Streak reset = 1	current_streak = 1	Pass
T4	3-day milestone	Celebration shown	Confetti modal displayed	Pass
T5	Same day msg	Count +1, streak same	Correct behavior	Pass
T6	Calendar update	Green block today	Heatmap updated	Pass

Table 5.5: Test Cases for Streak Tracking

5.2.6 Voice Features

ID	Scenario	Expected	Actual	Result
T1	Mic: say "hello"	hello in input	Transcribed correctly	Pass
T2	Say "Clock."	"Clock" (no period)	Period stripped	Pass
T3	Click speaker	Message read aloud	TTS speaks message	Pass
T4	Auto-play on	Auto-spoken	Messages spoken on receive	Pass
T5	Rate 1.5x	Faster speech	Speech rate increased	Pass

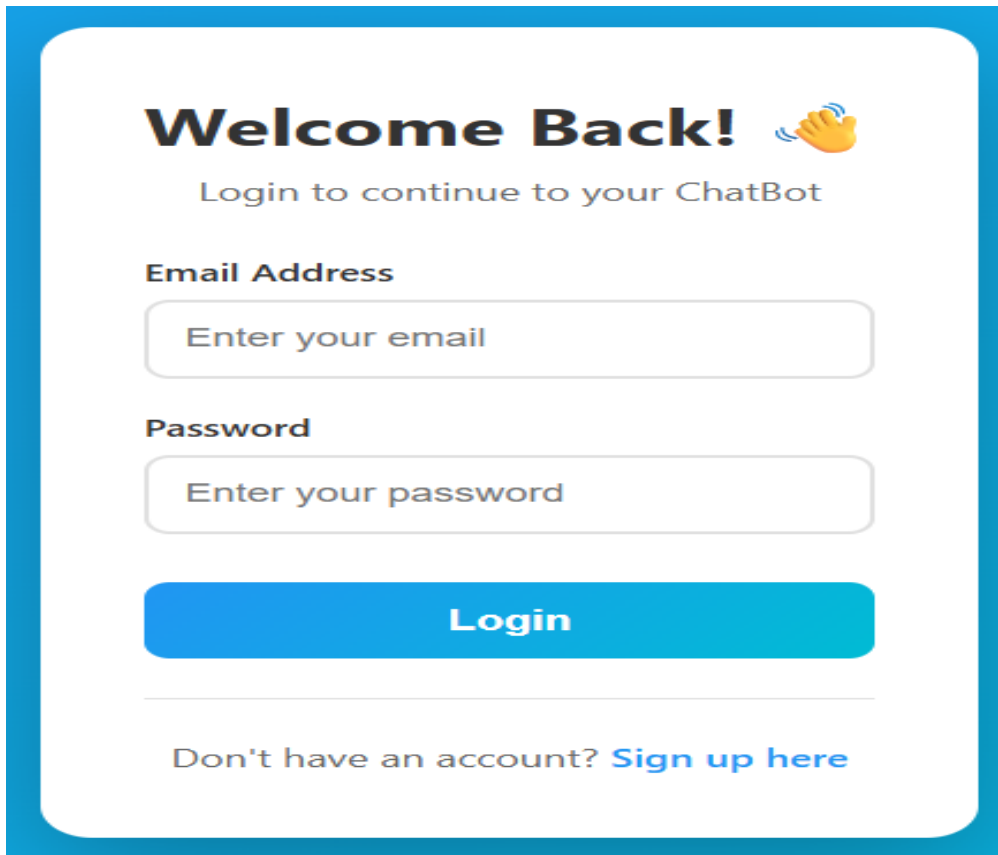
Table 5.6: Test Cases for Voice Features

6. Screenshots

6.1 Output Screenshots for Each Feature

6.1.1 Login & Signup Screens

When a user opens the application for the first time, they are presented with the Login screen with email and password fields. New users can navigate to the Signup page to create an account with email, username, and password. The authentication pages feature a gradient-themed background with smooth animations.

A login screen mockup with a blue gradient background. The screen features a white rounded rectangle containing the text "Welcome Back!" in bold black font, followed by a yellow hand emoji. Below this is the text "Login to continue to your ChatBot" in a smaller black font. There are two input fields: "Email Address" with the placeholder "Enter your email" and "Password" with the placeholder "Enter your password". A blue "Login" button is positioned below the password field. At the bottom, there is a link that says "Don't have an account? Sign up here".

Welcome Back! 🙌

Login to continue to your ChatBot

Email Address

Enter your email

Password

Enter your password

Login

Don't have an account? [Sign up here](#)

Fig. 6.2: LoginScreen

Create Account

Sign up to get started with ChatBot

Email Address

Username

Password

Confirm Password

Sign Up

Already have an account? [Login here](#)

Fig. 6.2: Signup Screen

6.1.2 Dashboard - Welcome Screen

After login, the user sees the Dashboard with a Welcome message and their username. The header displays the streak badge showing current streak and progress to next milestone. The notification bell shows unseen notification count.

Suggested prompts are shown for quick interaction.

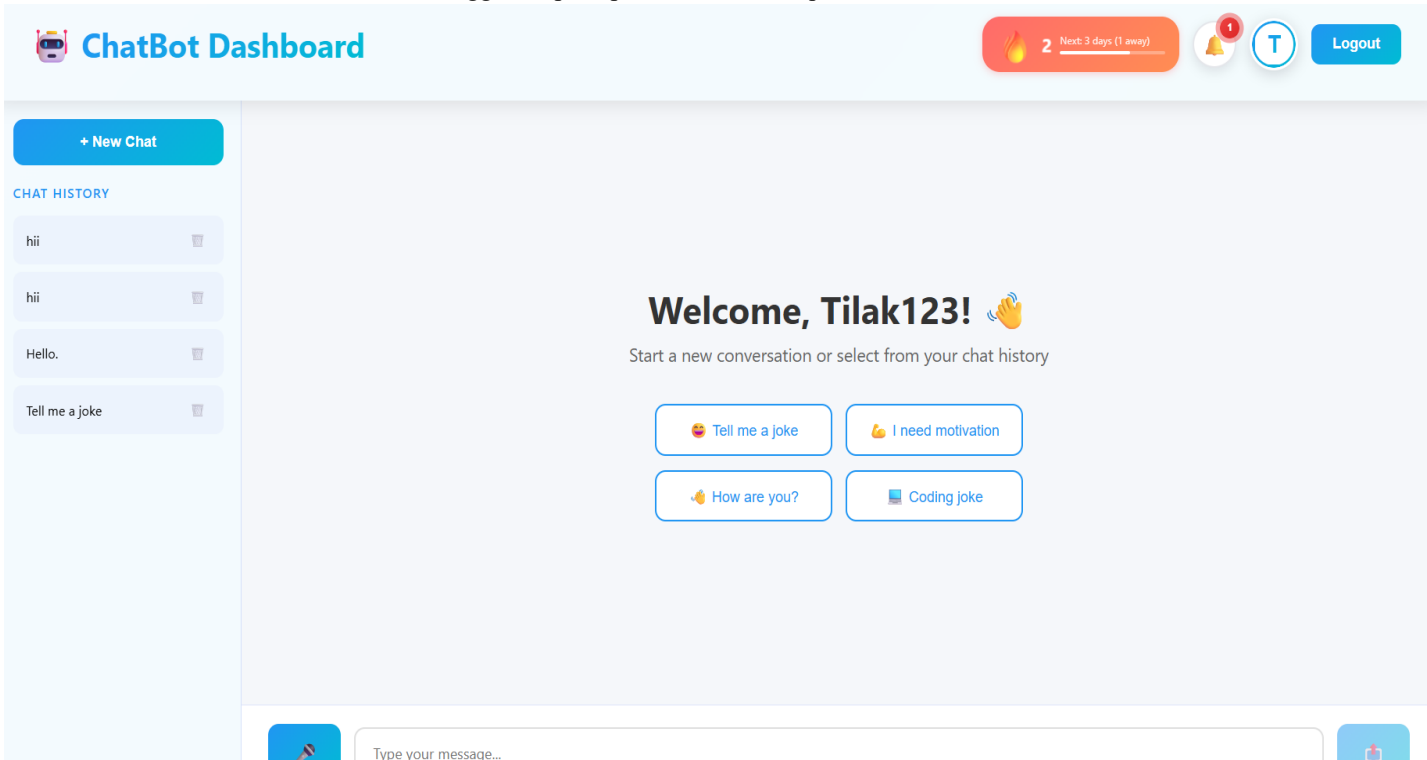


Fig. 6.3: Dashboard - Welcome Screen

6.1.3 Chat Conversation

User messages appear on the right side with gradient bubbles. Bot responses appear on the left with robot emoji avatar. Each message shows a timestamp. Bot messages include a speaker button for text-to-speech. Spell correction notice appears when corrections are applied. Chat sidebar shows chat history with delete buttons.

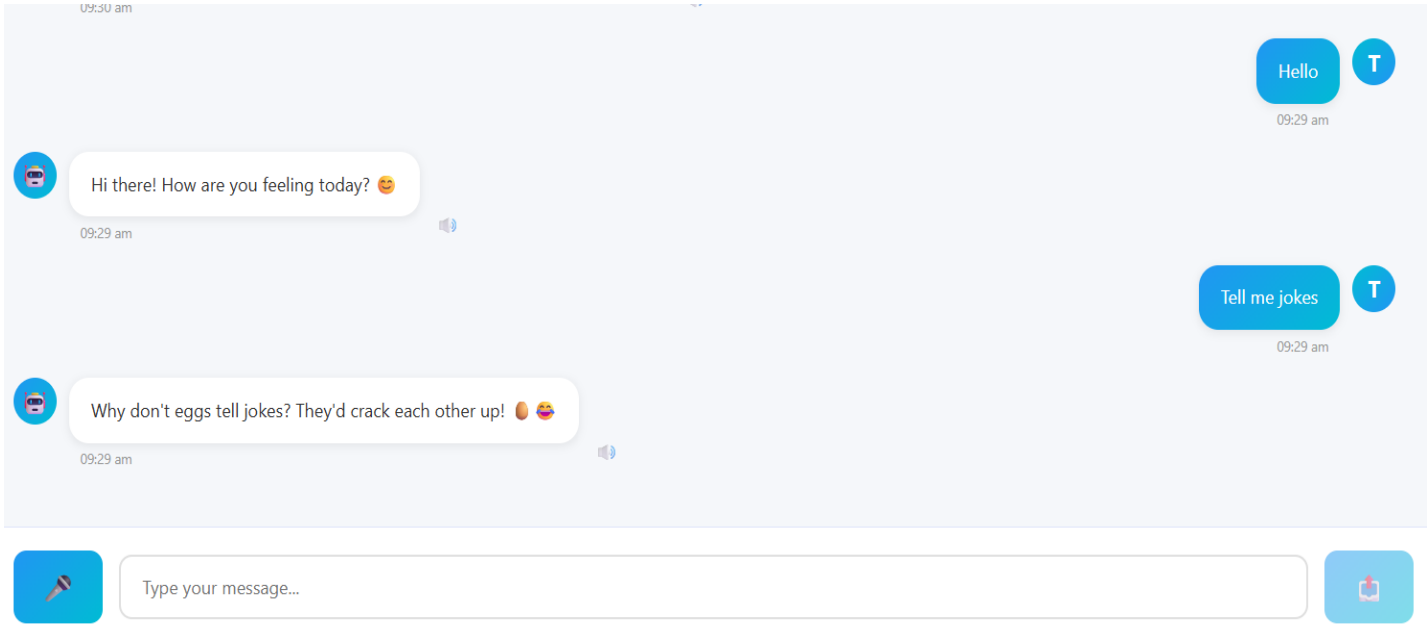


Fig. 6.4: Chat Conversation

6.1.4 Game Sessions

Quiz displays questions with 4 options (A/B/C/D), shows correct/incorrect feedback with explanations, and a final score. Riddle shows riddle text, accepts answers, provides hints on request. Word Scramble shows scrambled word, tracks attempts, provides hints, and awards bonus for first-try correct answers.

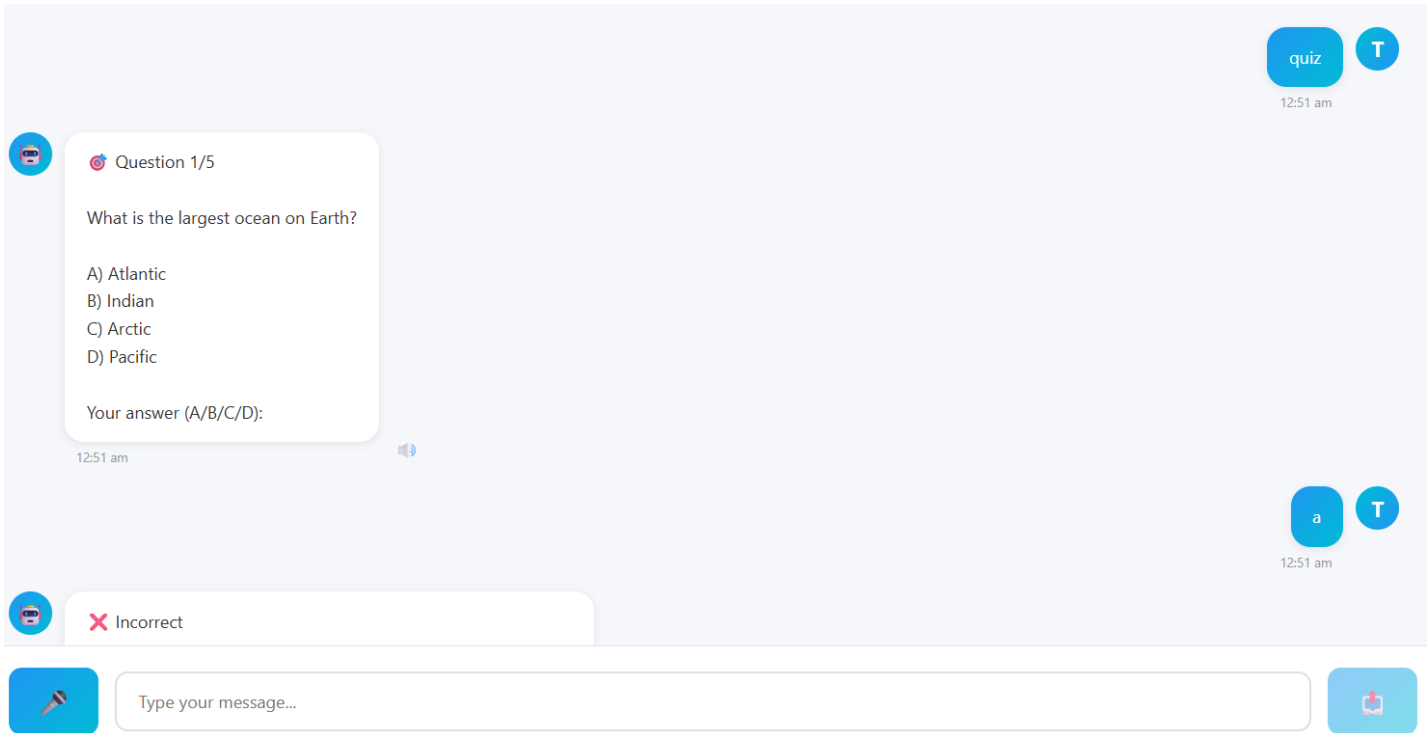


Fig. 6.5: Game Session (Quiz)

6.1.5 Profile Page with Streak Stats

User info section shows username, email, and member since date. Streak statistics displayed in a 2x2 grid: Current Streak, Longest Streak, Total Active Days, Next Milestone. Achievements section shows earned badges with emojis

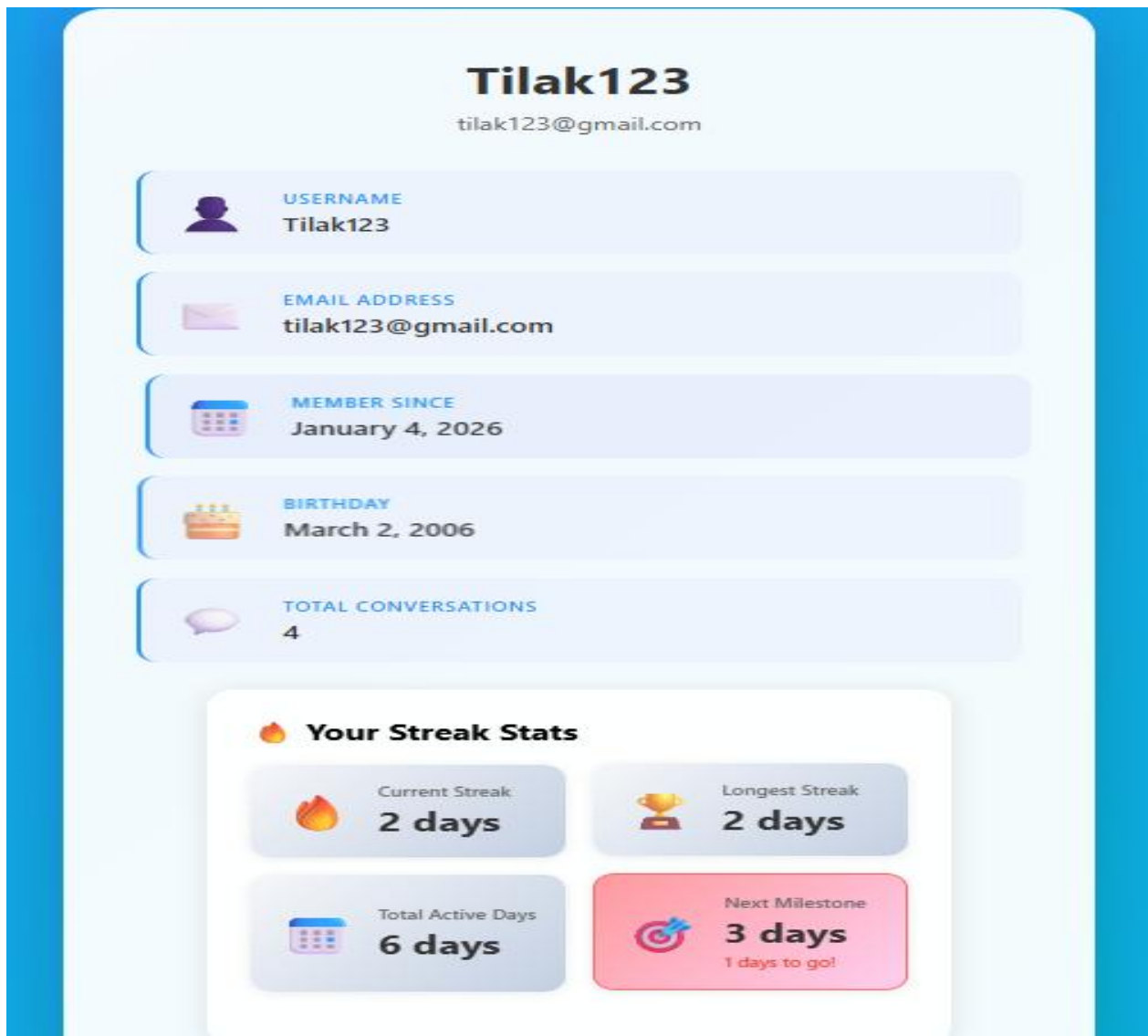


Fig. 6.6: Profile Page with Streak Stats

6.1.6 Activity Calendar Heatmap

GitHub-style contribution calendar showing the last 6 months of activity. 5 intensity levels from gray (no activity) to darkest green (20+ messages). Month labels centered above their respective columns. Hover tooltips show exact date and message count.

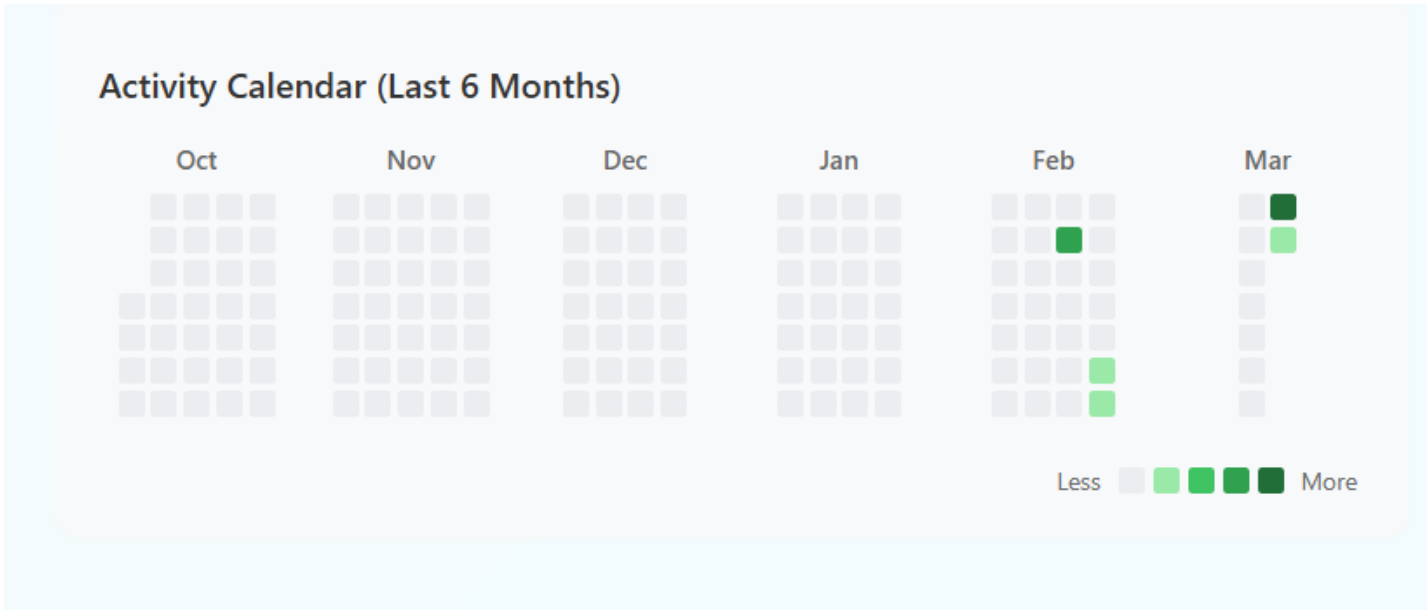


Fig. 6.7: Activity Calendar Heatmap

6.1.7 Notification System

Notification Panel: Dropdown from the bell icon showing all notifications with type-specific icons and priority coloring. Toast Notification: Auto-dismissing popup shown once per day for unseen notifications. Desktop Notification: Browser notification for

high-priority items (birthday, milestone).

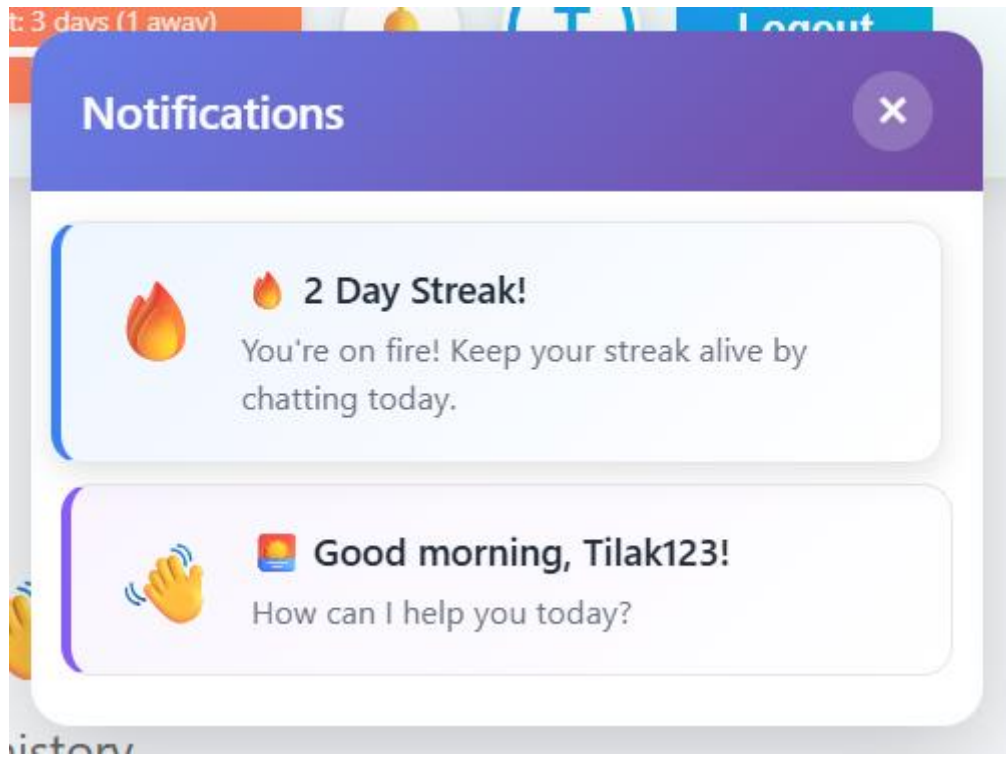


Fig. 6.8: Notification Panel

6.1.8 Milestone Celebration

Full-screen modal with confetti animation (50 pieces) triggered when user reaches a streak milestone (3, 7, 14, 30, 60, 100 days).

Displays milestone name, badge emoji, and congratulatory message.

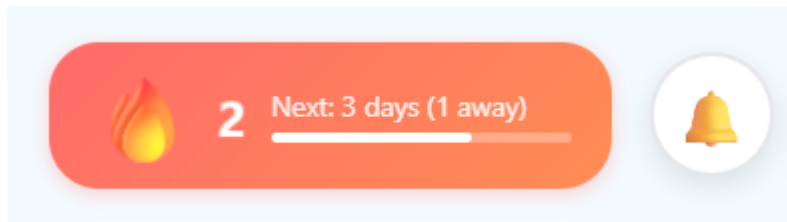


Fig. 6.9: Milestone Celebration

6.1.9 Theme & Voice Settings

Theme Selection Modal: 6 gradient themes -- Purple Galaxy, Ocean Blue, Sunset Orange, Forest Green, Rose Pink, Midnight Dark.

Voice Settings Modal: Toggle voice on/off, toggle auto-play, select voice from available browser voices, adjust speech rate and pitch

with sliders.

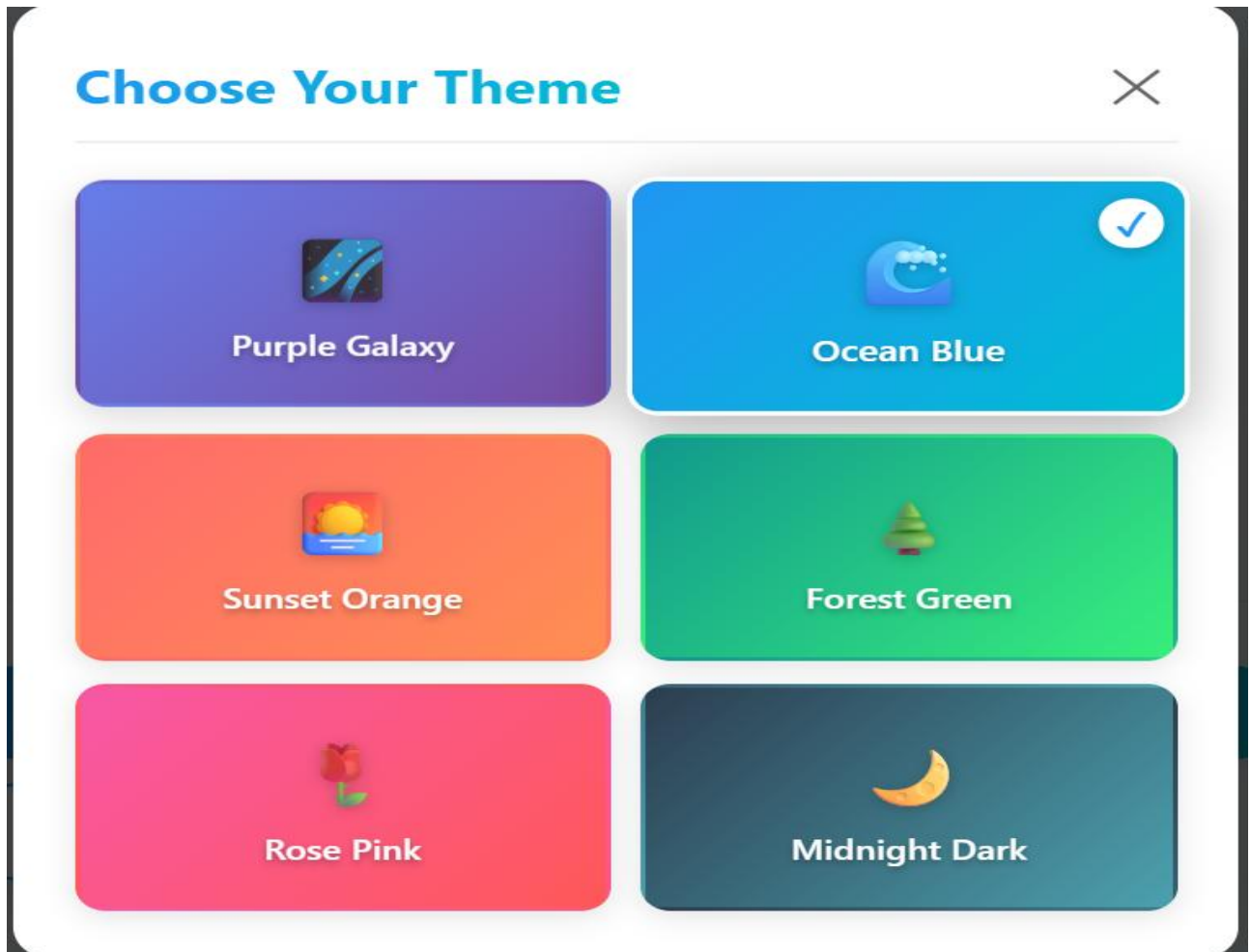


Fig. 6.10: Theme Selection

Voice Settings



Enable Voice Features

Turn on voice input and output



Auto-play Responses

Automatically speak bot messages



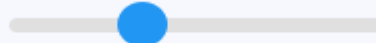
Voice

Choose
a voice

Microsoft David - English (United States) (en-US) ▼

Speed

Adjust speaking speed (1x)



Pitch

Adjust voice pitch (1.3x)



Fig. 6.11: Voice Settings

7. Conclusion

This project aimed at developing a Personalized Chatbot to Entertain the Users -- a web-based conversational application that combines natural language processing, semantic search, interactive games, voice interaction, and engagement tracking into a cohesive user experience.

The chatbot's intelligence is entirely custom-built with no dependency on external LLM APIs such as OpenAI or Google Gemini. Instead, we implemented a multi-layered NLP pipeline consisting of:

- Intent classification using regex and keyword matching
- Entity extraction for topics, emotions, people, and behaviors
- Dynamic response generation using template formatting with contextual advice
- Semantic search fallback using FAISS vector indexing with sentence-transformer embeddings(all-MiniLM-L6-v2)
- 5-tier fallback system ensuring the bot always provides a meaningful response

Beyond conversational AI, the project includes engaging features such as:

- Interactive games (quiz, riddle, word scramble, trivia, jokes) with persistent per-chat game state
- Voice input/output using the Web Speech API for hands-free interaction
- Daily streak tracking with milestone celebrations featuring confetti animations
- Activity calendar providing a visual GitHub-style heatmap of user engagement
- Smart notifications that respect user attention by only showing unseen items

The backend is developed using Flask 3.0 with PostgreSQL and SQLAlchemy, while the frontend uses React 18 with a component-based architecture, Context API for state management, and CSS gradient themes for visual customization.

After performing comprehensive testing with unit tests, integration tests, and black box test cases across all features, we conclude that the project is working successfully and provides a smooth, emotionally intelligent conversational experience.

"Technology is best when it brings people together." -- Matt Mullenweg

8. Limitation and Future Extension

8.1 Limitations

1. Rule-based intent classification -- The intent classification relies on regex patterns and keyword matching, which may miss nuanced expressions, sarcasm, or complex multi-intent messages.
2. Limited language support -- The system currently supports English only for the knowledge base, intent classification, and entity extraction.
3. Fixed knowledge base -- The chatbot's knowledge is limited to the pre-loaded knowledge_emotional.txt file. It cannot learn from new conversations or update its knowledge dynamically.
4. Browser dependency for voice -- Voice input/output depends on browser-native Web Speech API, which has varying support and quality across different browsers and platforms.
5. No real-time typing indicators -- The server does not use WebSocket for real-time communication; responses are delivered after full processing via HTTP.

8.2 Future Extensions

1. Integration with LLMs -- Optionally integrate with large language models (e.g., OpenAI GPT, Google Gemini) for more nuanced and context-aware conversations while keeping the local pipeline as fallback.
2. Multi-language support -- Extend the knowledge base, intent classification, and entity extraction to support multiple languages including Hindi, Gujarati, and other Indian languages.
3. Sentiment analysis improvement -- Implement transformer-based sentiment analysis models for more accurate emotion detection beyond keyword matching.
4. Conversation summarization -- Add the ability to summarize past conversations and carry emotional context across chat sessions.
5. WebSocket integration -- Replace HTTP polling with WebSocket for real-time bidirectional communication, enabling typing indicators and instant message delivery.
6. Mobile application -- Develop React Native or Flutter mobile applications to extend the chatbot's accessibility to mobile platforms.
7. Admin dashboard -- Create an admin panel for managing the knowledge base, monitoring user engagement metrics, and analyzing conversation patterns.
8. Adaptive learning -- Implement feedback loops where user reactions (positive/negative) help improve response quality over time.

9. Bibliography

The following links and websites are referred during the development of this project:

9.1 NLP and AI

Description	References
Sentence Transformers Docs	https://www.sbert.net/
FAISS (Facebook AI Search)	https://github.com/facebookresearch/faiss
all-MiniLM-L6-v2 Model	https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
pyspellchecker Documentation	https://pyspellchecker.readthedocs.io/
NLP Intent Classification	https://www.geeksforgeeks.org/intent-classification-in-nlp/
Python Regex Documentation	https://docs.python.org/3/library/re.html

Table 9.1: References for NLP and AI

9.2 Frontend Development

Description	References
React 18 Documentation	https://react.dev/
React Router v6 Docs	https://reactrouter.com/
Axios HTTP Client	https://axios-http.com/
Web Speech API (MDN)	https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
Web Notifications API	https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API
CSS Custom Properties	https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

Table 9.2: References for Frontend Development

9.3 Backend Development

Description	References
Flask Documentation	https://flask.palletsprojects.com/
Flask-JWT-Extended	https://flask-jwt-extended.readthedocs.io/
Flask-SQLAlchemy	https://flask-sqlalchemy.palletsprojects.com/
Flask-Bcrypt	https://flask-bcrypt.readthedocs.io/
PostgreSQL Documentation	https://www.postgresql.org/docs/
SQLAlchemy ORM Docs	https://docs.sqlalchemy.org/
Flask-CORS	https://flask-cors.readthedocs.io/

Table 9.3: References for Backend Development

9.4 General References

Description	References
UML Diagrams Guide	https://www.uml-diagrams.org/
Google Search	https://www.google.com/
StackOverflow	https://stackoverflow.com/
GitHub	https://github.com/

Table 9.4: References for Project Report